Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**

ОТЧЕТ

по лабораторной работе № 5

Название: Работа с веб-страницами с помощью библиотеки React
Дисциплина: Языки интернет программирования

Студент ИУ6-64Б (Группа) _____ (Подпись, дата) Д. О. Романов (И.О. Фамилия)

Преподаватель _____ (Подпись, дата) В. И. Кузнецов (И.О. Фамилия)

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Цель работы – знакомство с библиотекой React, освоение основных концепций: компоненты, состояние, пропсы, хуки, управление формами и списками.

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

React – JavaScript-библиотека для создания пользовательских интерфейсов. Основана на компонентном подходе. Компоненты могут быть функциональными, используют хуки для управления состоянием и побочными эффектами. Виртуальный DOM повышает производительность за счёт минимальных изменений реального DOM. Основные хуки: useState для состояния, useEffect для эффектов. JSX позволяет писать HTML-подобный код внутри JavaScript.

3. ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1. Задание 1. Счётчик-кликер

Создан компонент Counter с состоянием count. Реализованы кнопки +1, –1 (блокируется при 0), Reset. Добавлено сообщение при кратности 10. Поддержаны горячие клавиши стрелок вверх/вниз.

Листинг 1 - код всех заданий

```
<!DOCTYPE html>
<html lang="ru">
<head>
   <meta charset="UTF-8">
   <title>все вместе</title>
                                                                <script
src="https://unpkg.com/react@18/umd/react.development.js"></script>
                                                                <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"><
/script>
                                                                <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
   <style>
            body { font-family: Arial, sans-serif; padding: 20px;
max-width: 600px; margin: 0 auto; }
       .nav { display: flex; gap: 10px; margin-bottom: 20px; }
             .nav button { padding: 8px 16px; cursor: pointer;
background: #f0f0f0; border: 1px solid #ccc; border-radius: 4px; }
          .nav button.active { background: #007bff; color: white;
border-color: #0056b3; }
       button { padding: 5px 10px; margin: 0 5px; }
       .congrats { color: green; font-weight: bold; }
       .completed { text-decoration: line-through; color: gray; }
       ul { list-style: none; padding: 0; }
       li { margin: 8px 0; }
       .filter-btn { margin: 0 5px; padding: 5px 10px; }
       .active-filter { background-color: lightblue; }
         .progress-bar { width: 100%; background: #f0f0f0; height:
20px; border-radius: 10px; margin: 10px 0; }
             .progress-fill { height: 100%; background: #4caf50;
border-radius: 10px; transition: width 0.3s; }
       .options { margin: 15px 0; }
       .option { display: block; margin: 5px 0; }
   </style>
</head>
<body>
   <div id="root"></div>

   <script type="text/babel">
       function Counter() {
           const [count, setCount] = React.useState(0);

           const increment = () => setCount(c => c + 1);
            const decrement = () => setCount(c => (c > 0 ? c - 1 :
0));
           const reset = () => setCount(0);
```

```
            React.useEffect(() => {
                const handleKeyDown = (e) => {
                    if (e.key === 'ArrowUp') {
                        e.preventDefault();
                        increment();
                    } else if (e.key === 'ArrowDown') {
                        e.preventDefault();
                        decrement();
                    }
                };
                window.addEventListener('keydown', handleKeyDown);
                  return () => window.removeEventListener('keydown',
handleKeyDown);
            }, []);

            const showCongrats = count > 0 && count % 10 === 0;

            return (
                <div style={{ textAlign: 'center' }}>
                    <h1>счетчик: {count}</h1>
                    <button onClick={increment}>+1</button>
                     <button onClick={decrement} disabled={count ===
0}>−1</button>
                    <button onClick={reset}>сброс</button>
                     {showCongrats && <p className="congrats">кратно
10!</p>}
                                    <p><small>стрелки  вверх/вниз  тоже
работают</small></p>
                </div>
            );
        }

        function TodoApp() {
            const [tasks, setTasks] = React.useState([]);
            const [inputValue, setInputValue] = React.useState('');
            const [filter, setFilter] = React.useState('all');

            React.useEffect(() => {
                const saved = localStorage.getItem('tasks');
                if (saved) setTasks(JSON.parse(saved));
            }, []);

            React.useEffect(() => {
                                        localStorage.setItem('tasks',
JSON.stringify(tasks));
```

```
            }, [tasks]);

            const addTask = () => {
                if (inputValue.trim() === '') return;
                const newTask = {
                    id: Date.now(),
                    text: inputValue,
                    completed: false
                };
                setTasks([...tasks, newTask]);
                setInputValue('');
            };

            const handleKeyPress = (e) => {
                if (e.key === 'enter') addTask();
            };

            const toggleTask = (id) => {
                setTasks(tasks.map(task =>
                            task.id === id ? { ...task, completed:
!task.completed } : task
                ));
            };

            const deleteTask = (id) => {
                setTasks(tasks.filter(task => task.id !== id));
            };

            const filteredTasks = tasks.filter(task => {
                if (filter === 'active') return !task.completed;
                if (filter === 'completed') return task.completed;
                return true;
            });

            const total = tasks.length;
                        const completedCount = tasks.filter(t =>
t.completed).length;

            return (
                <div>
                    <h1>todo</h1>
                    <div>
                        <input
                            type="text"
                            value={inputValue}
```

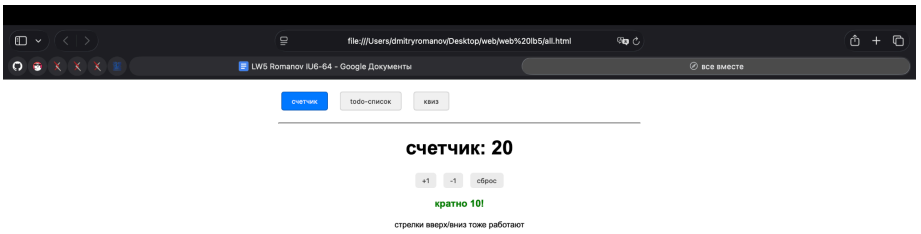
```
                                                        onChange={(e) ⇒
setInputValue(e.target.value)}
                            onKeyPress={handleKeyPress}
                            placeholder="Новая задача"
                        />
                        <button onClick={addTask}>Добавить</button>
                    </div>

                    <div style={{ margin: '15px 0' }}>
                        <button
                                 className={`filter-btn ${filter ===
'all' ? 'active-filter' : ''}`}
                            onClick={() ⇒ setFilter('all')}
                        >все</button>
                        <button
                                 className={`filter-btn ${filter ===
'active' ? 'active-filter' : ''}`}
                            onClick={() ⇒ setFilter('active')}
                        >активные</button>
                        <button
                                 className={`filter-btn ${filter ===
'completed' ? 'active-filter' : ''}`}
                            onClick={() ⇒ setFilter('completed')}
                        >выполненные</button>
                    </div>

                                      <p>всего: {total}, выполнено:
{completedCount}</p>

                    <ul>
                        {filteredTasks.map(task ⇒ (
                            <li key={task.id}>
                                <input
                                    type="checkbox"
                                    checked={task.completed}
                                                     onChange={() ⇒
toggleTask(task.id)}
                                />
                                   <span className={task.completed ?
'completed' : ''}>
                                    {task.text}
                                </span>
                                             <button onClick={() ⇒
deleteTask(task.id)}>удалить</button>
                            </li>
                        ))}
```

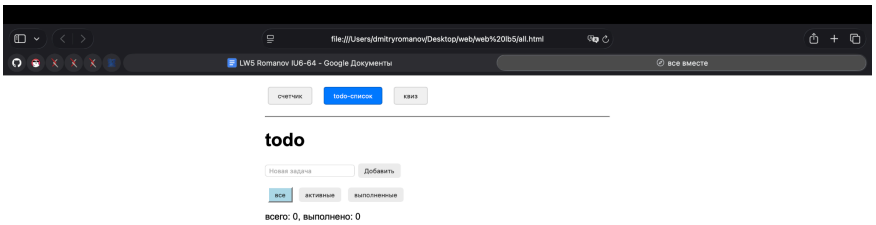
```
                    </ul>
                </div>
            );
        }

        const questions = [
            {
                        question: "какая библиотека используется для
создания пользовательских интерфейсов?",
                options: ["Django", "React", "Flask", "Laravel"],
                correctIndex: 1
            },
            {
                    question: "какой хук используется для состояния в
функциональном компоненте?",
                     options: ["useEffect", "useState", "useContext",
"useReducer"],
                correctIndex: 1
            },
            {
                question: "что такое JSX?",
                options: [
                    "расширение JavaScript для XML",
                    "название фреймворка",
                    "библиотека для стилей",
                    "язык программирования"
                ],
                correctIndex: 0
            },
            {
                question: "для чего нужны ключи (keys) в списках?",
                options: [
                    "для красоты",
                    "для идентификации элементов при обновлении",
                    "для сортировки",
                    "для стилизации"
                ],
                correctIndex: 1
            },
            {
                     question: "какой метод используется для рендера
React-приложения в DOM?",
                        options: ["ReactDOM.render", "React.render",
"createRoot", "render"],
                correctIndex: 2
            }
```

```
        ];

        function Quiz() {
                          const  [currentIndex,  setCurrentIndex]  =
React.useState(0);
            const [answers, setAnswers] = React.useState([]);
                             const  [showResult,  setShowResult]  =
React.useState(false);

            const currentQuestion = questions[currentIndex];
            const selectedAnswer = answers[currentIndex];

            const handleSelect = (index) => {
                const newAnswers = [...answers];
                newAnswers[currentIndex] = index;
                setAnswers(newAnswers);
            };

            const next = () => {
                if (currentIndex < questions.length - 1) {
                    setCurrentIndex(currentIndex + 1);
                } else {
                    setShowResult(true);
                }
            };

            const restart = () => {
                setCurrentIndex(0);
                setAnswers([]);
                setShowResult(false);
            };

            const correctCount = answers.reduce((acc, ans, idx) => {
                return ans === questions[idx].correctIndex ? acc + 1
: acc;
            }, 0);

                    const  progressPercent  =  ((currentIndex  +  1)  /
questions.length) * 100;

            if (showResult) {
                return (
                    <div style={{ textAlign: 'center' }}>
                        <h2>результат</h2>
                          <p>вы ответили правильно на {correctCount}
из {questions.length}</p>
```

```
                                <button onClick={restart}>пройти ещё
раз</button>
                    </div>
                );
            }

            return (
                <div>
                                    <h2>вопрос {currentIndex + 1} /
{questions.length}</h2>
                    <div className="progress-bar">
                             <div className="progress-fill" style={{
width: `${progressPercent}%` }}></div>
                    </div>


<p><strong>{currentQuestion.question}</strong></p>

                    <div className="options">
                          {currentQuestion.options.map((option, idx)
⇒ (
                            <label key={idx} className="option">
                                <input
                                    type="radio"
                                    name="option"
                                    value={idx}
                                    checked={selectedAnswer === idx}
                                                    onChange={() ⇒
handleSelect(idx)}
                                />
                                {option}
                            </label>
                        ))}
                    </div>

                    <button
                        onClick={next}
                        disabled={selectedAnswer === undefined}
                    >
                            {currentIndex === questions.length - 1 ?
'Завершить' : 'Далее'}
                    </button>
                </div>
            );
        }
```

```
       function App() {
                            const  [currentTask,  setCurrentTask]  =
React.useState('counter');

           return (
               <div>
                   <div class="nav">
                       <button
                             class={currentTask === 'counter' ?
'active' : ''}
                                                        onClick={()  =>
setCurrentTask('counter')}
                       >
                           счетчик
                       </button>
                       <button
                                class={currentTask === 'todo' ?
'active' : ''}
                           onClick={() => setCurrentTask('todo')}
                       >
                           todo-список
                       </button>
                       <button
                                class={currentTask === 'quiz' ?
'active' : ''}
                           onClick={() => setCurrentTask('quiz')}
                       >
                           квиз
                       </button>
                   </div>

                   <hr />

                   {currentTask === 'counter' && <Counter />}
                   {currentTask === 'todo' && <TodoApp />}
                   {currentTask === 'quiz' && <Quiz />}
               </div>
           );
       }

                                            const      root      =
ReactDOM.createRoot(document.getElementById('root'));
       root.render(<App />);
   </script>
</body>
</html>
```





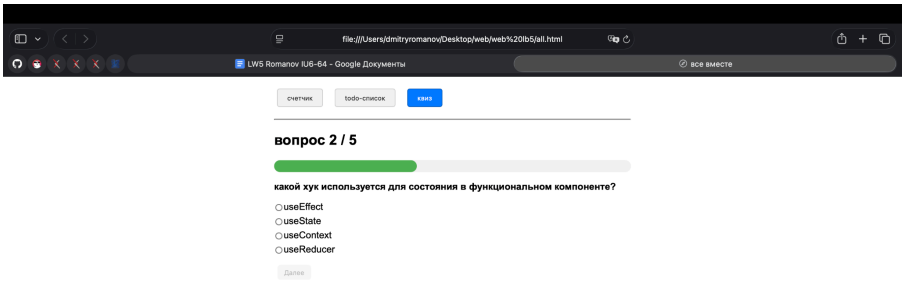


Рисунок 1,2,3 – страница

4. ВЫВОДЫ

В ходе работы изучены основные возможности React: создание компонентов, работа с состоянием, обработка событий, условный рендеринг, списки и ключи, использование эффектов для синхронизации с localStorage. Полученные навыки позволяют разрабатывать интерактивные веб-интерфейсы.

5. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Компонент – это независимый блок интерфейса; состояние – внутренние данные, меняющиеся со временем; пропсы – внешние данные, передаваемые родителем.
2. JSX – расширение синтаксиса JavaScript для описания разметки, преобразуемое в React.createElement.
3. React вызывает функцию компонента, получает React-элементы, строит виртуальный DOM, сравнивает с предыдущим и обновляет реальный DOM.
4. Компонент обновляется при изменении state, props или перерисовке родителя (даже если props не менялись).
5. Использованы хуки: useState (для хранения данных), useEffect (для работы с localStorage и клавишами).
6. Основные правила React: хуки вызываются только на верхнем уровне; не изменять состояние напрямую; компоненты должны быть чистыми функциями; ключи в списках должны быть стабильными и уникальными.